SGD with Momentum, and Adagrad

1. Why Plain Gradient Descent Struggles

Gradient Descent updates parameters by taking a step opposite to the gradient of the loss function J(θ) with respect to the parameters θ, scaled by a learning rate η:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta J(\theta_t)$$

Eq. 1 — Vanilla (batch/stochastic) gradient descent update

This works, but on real loss surfaces — especially ones that are elongated or 'ravine-shaped', where curvature differs sharply between directions — plain SGD suffers from two related problems:

- It oscillates back and forth across the steep direction of the surface instead of moving smoothly toward the minimum.
- Progress along the shallow (flat) direction is very slow, because each individual step is small and uncorrelated with previous steps.
- With mini-batches, the gradient itself is noisy, so the path can be erratic even near the minimum.

2. The Idea of Momentum

Momentum borrows an idea from physics: imagine a ball rolling down a bowl-shaped valley. Instead of instantly changing direction every time the local slope changes, the ball keeps some of its previous velocity — it 'remembers' the direction it has been moving in and builds up speed when gradients repeatedly point the same way.

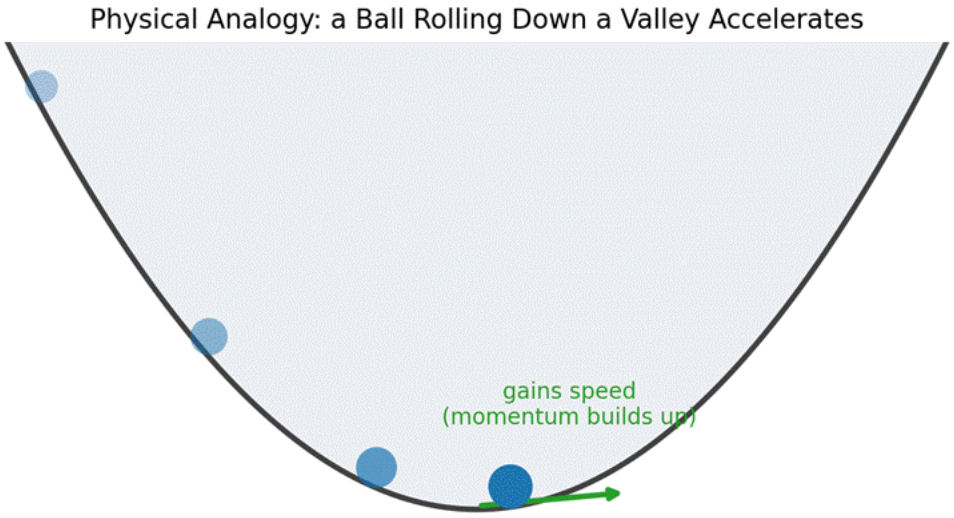


Fig. 1 — Physical analogy: a ball accumulates speed as it repeatedly moves in the same direction.

In optimization terms, instead of updating parameters directly with the raw gradient, we first update a velocity vector v that is an exponentially-weighted moving average of past gradients, and then use v to update the parameters.

3. Mathematical Formulation

3.1 Velocity (momentum) update

$$v_t = \beta v_{t-1} + \eta \nabla_\theta J(\theta_t)$$

Eq. 2 — Velocity update: blend of previous velocity and current gradient

where:

- v_t — velocity at step t (initialised as shown below)
- β — momentum coefficient, 0 ≤ β < 1 (commonly 0.9)
- η — learning rate
- ∇θ J(θt) — gradient of the loss at the current parameters

$$v_0 = 0$$

3.2 Parameter update

$$\theta_{t+1} = \theta_t - v_t$$

Eq. 3 — Parameters move along the accumulated velocity, not the raw gradient

3.3 Unrolling the recursion

Expanding Eq. 2 backward in time shows that v_t is really a weighted sum of all past gradients, with exponentially decaying weights β raised to increasing powers. This is exactly why momentum smooths the trajectory — recent gradients matter most, but older ones still contribute:

$$v_t = \eta \sum_{i=0}^{t} \beta^{t-i} \nabla_\theta J(\theta_i)$$

Eq. 4 — Momentum as an exponentially-weighted sum of historical gradients

3.4 Equivalent 'EWMA' form

Some textbooks (and frameworks such as TensorFlow) write the velocity update as a true exponentially-weighted moving average with weights that sum to 1:

$$v_t = \beta v_{t-1} + (1-\beta) g_t$$

Eq. 5 — Alternative normalized form (g_t denotes the gradient at step t)

Both forms are mathematically equivalent up to a rescaling of the learning rate; the difference is only in bookkeeping.

4. Why Momentum Works — Vector Intuition

At every step, the new velocity is a combination of two vectors: the previous velocity (scaled by β) and the current gradient step (scaled by η). When consecutive gradients point in similar directions, these vectors reinforce each other and the effective step size

grows. When gradients oscillate (point in alternating directions), the components cancel and the oscillation is damped.

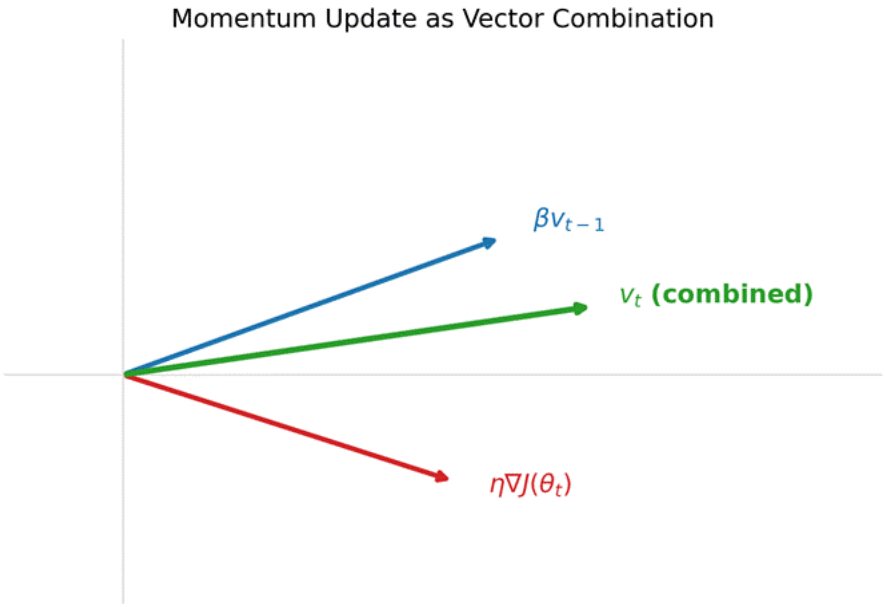


Fig. 2 — The new velocity is the vector sum of the decayed old velocity and the new gradient step.

5. Effective Learning Rate

Because velocity keeps accumulating in a consistent direction, momentum effectively increases the step size in that direction. At steady state (constant gradient), the velocity converges to:

$$\eta_{\text{eff}} \approx \frac{\eta}{1-\beta}$$

Eq. 6 — Effective learning rate amplification factor 1 / (1 − β)

For β = 0.9 this is a 10× amplification; for β = 0.99 it is 100×. This is why β close to 1 requires a correspondingly smaller η, or training can become unstable and overshoot the minimum.

$$\beta \in \{0.9,\ 0.95,\ 0.99\}$$

Typical values used in practice

6. Trajectory Comparison

The figure below simulates gradient descent on an ill-conditioned quadratic loss surface — steep along one axis, shallow along the other — a standard test case for illustrating momentum. The star marks the minimum.

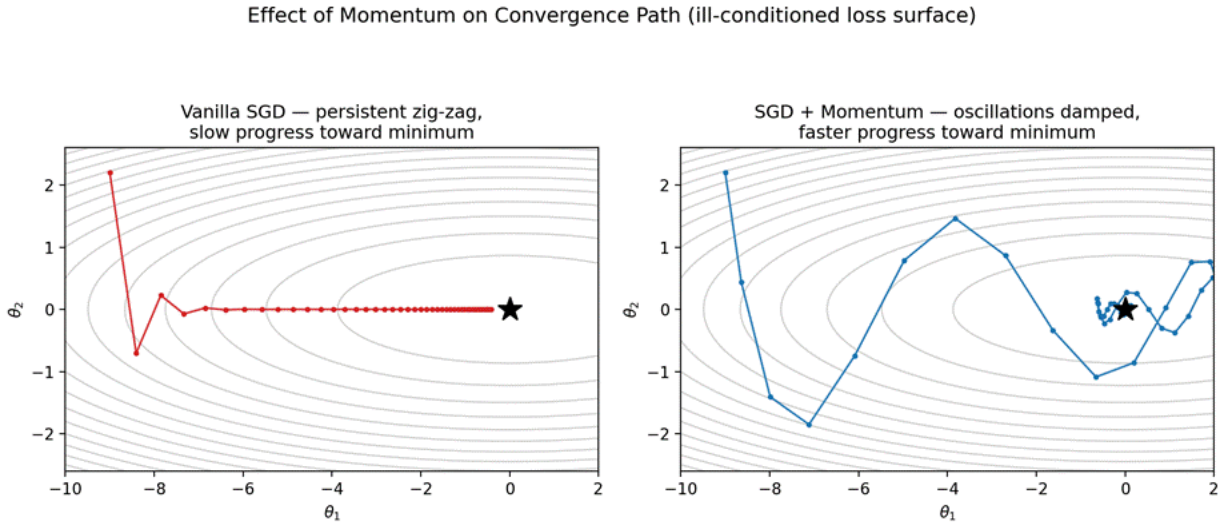


Fig. 3 — Vanilla SGD zig-zags and crawls slowly; momentum damps the oscillation and reaches the minimum region far sooner.

7. Nesterov Accelerated Gradient (NAG)

A refinement called Nesterov Momentum computes the gradient not at the current position θt, but at an approximated 'look-ahead' position — where momentum is already about to carry the parameters. This gives the optimizer a corrective, slightly anticipatory step:

$$\tilde{\theta}_t = \theta_t - \beta v_{t-1}$$

Eq. 7 — Look-ahead position

$$v_t = \beta v_{t-1} + \eta \nabla_\theta J(\tilde{\theta}_t)$$

Eq. 8 — Gradient evaluated at the look-ahead position

$$\theta_{t+1} = \theta_t - v_t$$

Eq. 9 — Parameter update (same form as classical momentum)

Intuitively, classical momentum first computes the gradient, then makes a big jump; Nesterov first jumps in the direction of accumulated momentum, then measures the gradient there and corrects. This lookahead tends to reduce overshoot and gives noticeably better convergence in practice, particularly for convex problems.

8. Vanilla SGD vs. SGD with Momentum — Summary

Aspect	Vanilla SGD	SGD with Momentum
Update rule	Uses only the current gradient	Uses an exponentially-weighted running average (velocity) of past gradients
Convergence in ravines	Zig-zags, very slow	Oscillations damped along steep axis, accelerated along shallow axis
Sensitivity to noisy gradients	High — each step can jump around	Lower — averaging smooths out noise
Extra hyper-parameter	None	Momentum coefficient β (typically 0.9–0.99)
Memory required	None beyond θ	One extra vector v, same shape as θ
Typical convergence speed	Baseline	Often 2–5× fewer iterations to converge
Risk	Getting stuck / slow crawl on plateaus	Can overshoot the minimum if β or η is too large

9. Advantages and Limitations of Momentum

Advantages

- Faster convergence, especially on ill-conditioned / ravine-shaped loss surfaces.
- Smooths out noisy gradient estimates from mini-batch training.
- Helps escape shallow local minima and saddle points due to accumulated velocity.
- Simple to implement — only one extra state vector per parameter.

Limitations

- Introduces a new hyper-parameter (β) that must be tuned alongside η.
- Can overshoot minima if momentum or learning rate is too high.
- Still sensitive to learning-rate scale; does not adapt per-parameter step sizes.

10. Adagrad — Adaptive Gradient Algorithm

Momentum solves the problem of a single, fixed learning rate applied the same way at every step. Adagrad tackles a different problem: what if different parameters need different learning rates? Some parameters (e.g. weights tied to rare / sparse features in text or recommendation data) receive gradient signal only occasionally and should take large steps when they do; others are updated on almost every example and should take smaller, more cautious steps.

Adagrad (Duchi et al., 2011) achieves this by keeping track, for every parameter individually, of the sum of squared gradients seen so far, and dividing the learning rate by the square root of that sum.

10.1 Gradient accumulator

$$g_t = \nabla_\theta J(\theta_t)$$

At every step, Adagrad accumulates the element-wise square of the gradient into a running total G (one entry per parameter):

$$G_t = G_{t-1} + g_t \odot g_t$$

Eq. 10 — Squared-gradient accumulator update (⊙ = element-wise product)

$$G_0 = 0$$

Unrolled, this is simply a running sum of every squared gradient seen for that parameter since training began:

$$G_t = \sum_{i=1}^{t} g_i \odot g_i$$

Eq. 11 — G_t is a cumulative, monotonically increasing sum

10.2 Parameter update

The standard learning rate η is then divided, per parameter, by the square root of its accumulator (plus a tiny ε to avoid division by zero):

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t + \epsilon}} \odot g_t$$

Eq. 12 — Adagrad parameter update

$$\epsilon \approx 10^{-8} \quad \text{(numerical stability)}$$

Because G is accumulated separately for every parameter, parameters that have historically received large or frequent gradients get their learning rate shrunk aggressively, while rarely-updated parameters keep a comparatively large effective learning rate.

10.3 Effective per-parameter learning rate

$$\eta_{\text{eff},\,i} = \frac{\eta}{\sqrt{G_{t,i} + \epsilon}}$$

Eq. 13 — Effective learning rate for parameter i at step t

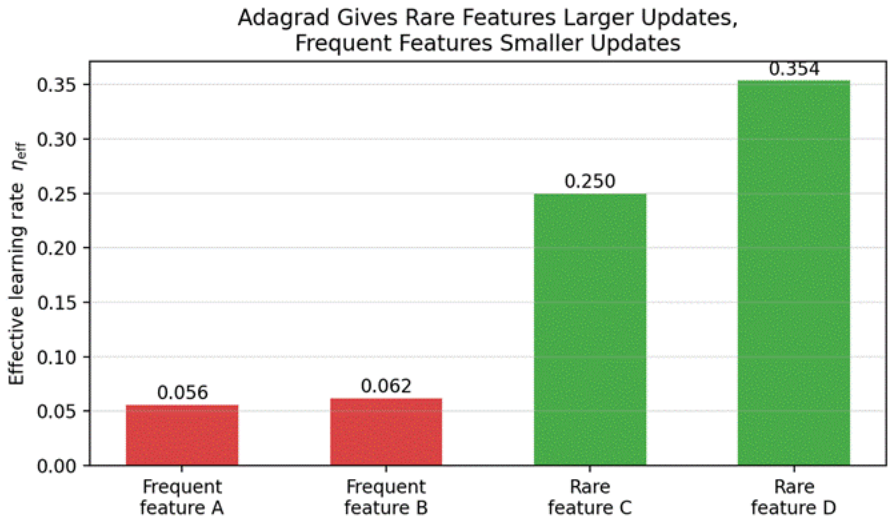


Fig. 4 — Illustrative effective learning rates: frequently-updated features end up with small steps, rare features keep large steps.

10.4 The learning rate decay problem

Since every term added to G is non-negative (it is a square), the accumulator can only grow — it never resets or decays. As training progresses, G keeps increasing, so the effective learning rate keeps shrinking, monotonically, for every parameter:

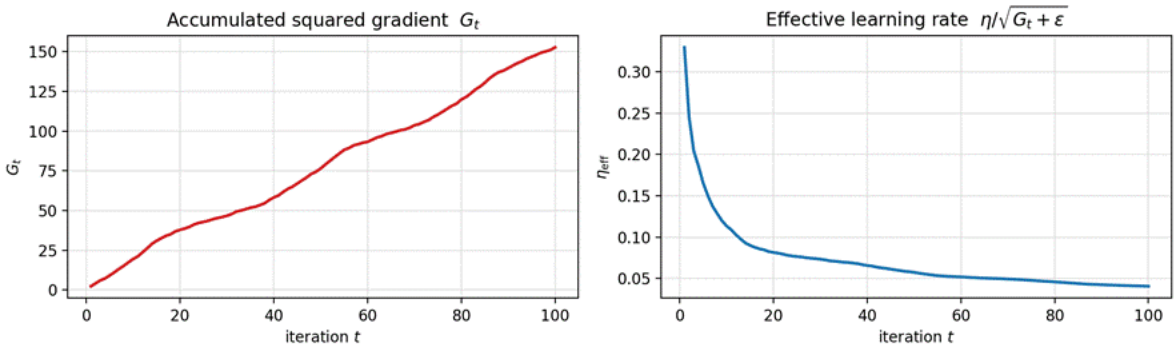


Fig. 5 — G_t grows without bound, so η_eff decays toward zero and training can stall on long runs.

This is Adagrad's best-known drawback: on long training runs the accumulated sum can become so large that the effective learning rate becomes vanishingly small, and learning effectively stops — even if the model has not yet reached a good minimum. This motivated later methods (RMSProp, Adadelta, Adam) that replace the ever-growing sum with an exponentially-decaying moving average instead, so old gradients are gradually 'forgotten'.

11. Trajectory Comparison — All Three Methods

Running vanilla SGD, SGD with Momentum, and Adagrad from the same starting point on the same ill-conditioned loss surface highlights their different characters: Adagrad's per-parameter scaling lets it take a large corrective step early (since the steep direction's accumulator grows quickly and shrinks its step, while the shallow direction keeps a larger step), converging efficiently without needing a hand-tuned momentum term.

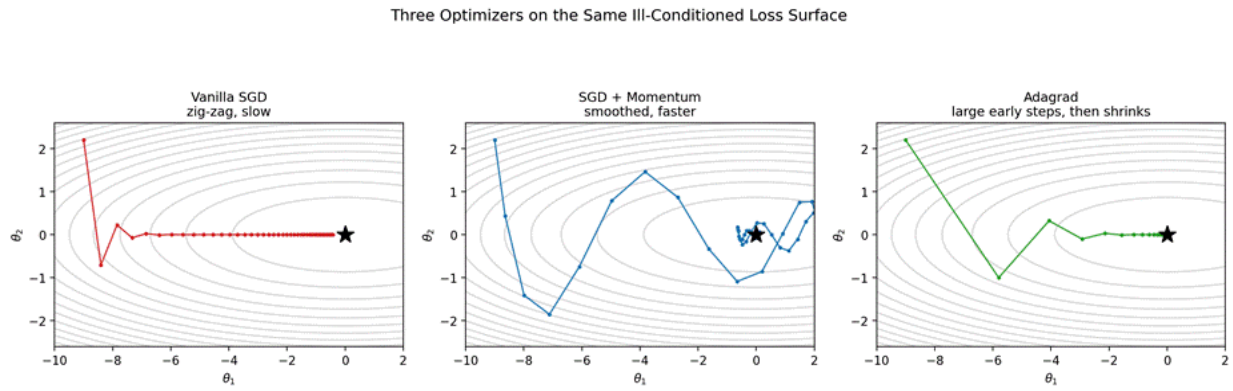


Fig. 6 — Vanilla SGD zig-zags; Momentum smooths the path; Adagrad adapts its step size per axis automatically.

12. Three-Way Summary

Aspect	Vanilla SGD	SGD + Momentum	Adagrad
Core idea	Step opposite the raw gradient	Step opposite a running average (velocity) of gradients	Step opposite gradient, scaled per-parameter by history of squared gradients
Learning rate	Fixed, same for all parameters	Fixed, same for all parameters	Adaptive — automatically shrinks per parameter
Extra state	None	Velocity vector v	Accumulator G (sum of squared gradients)
Extra hyper-parameter	None	Momentum β	None (only small ε for stability)
Good for	Simple, well-conditioned problems	Ravine-shaped / ill-conditioned losses	Sparse data, e.g. NLP / text features
Main weakness	Slow on ravines, sensitive to noise	Extra tuning; can overshoot	Learning rate can shrink to ≈ 0 and stall training

13. Adagrad — Advantages and Limitations

Advantages

- No manual per-parameter learning-rate tuning — it adapts automatically.
- Well suited to sparse data (NLP, recommendation systems, sparse one-hot features).

- Larger updates for infrequent parameters help them 'catch up' faster.
- Simple to implement, no momentum hyper-parameter to tune.

Limitations

- The accumulator G only grows, so the effective learning rate decays monotonically and can vanish on long training runs.
- Can stop learning prematurely on deep-learning tasks that need many iterations.
- Superseded in most deep-learning practice by RMSProp, Adadelta, and Adam, which fix the decay problem while keeping the adaptive idea.

14. Key Takeaways

- Momentum replaces the raw gradient with an exponentially-weighted running average of past gradients (the ‘velocity’), damping oscillations and accelerating convergence.
- The momentum coefficient β controls how much history is retained; higher β = smoother but risks overshoot. Nesterov momentum improves on this with a look-ahead gradient.
- Adagrad instead adapts the learning rate per parameter, using the accumulated sum of squared gradients — large steps for rarely-updated parameters, small steps for frequently-updated ones.
- Adagrad's accumulator never shrinks, so its learning rate decays monotonically and can stall training on long runs — this is its central weakness.
- Both ideas — momentum (temporal averaging of gradients) and adaptive per-parameter scaling (Adagrad) — are combined in modern optimizers such as RMSProp, Adadelta, and Adam.